

P1759R1

Native handle from file streams

Published Proposal, 2019-07-29

This version:

<https://wg21.link/P1759R1>

Author:

[Elias Kosunen](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/eliaskosunen/wg21: P1759R1

Abstract

This paper proposes adding functionality to `fstream` and `filebuf` to retrieve the native file handle.

Table of Contents

- 1 Revision History**
 - 1.1 Revision 1 (Draft)
 - 1.2 Revision 0
- 2 Overview**
- 3 Motivation**
- 4 Scope**
- 5 Design Discussion**

5.1 Type of `native_handle_type`

5.2 Member function vs free function

5.3 Precondition

6 Impact On the Standard and Existing Code

7 Implementation

8 Prior Art

8.1 Discussion

8.2 Existing precedent for presence of `native_handle`

9 Technical Specifications

? Native handles [`file.native`]

10 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 1 (Draft)

- Make `native_handle_type` be standard layout
- Add precondition (`is_open() == true`) to `.native_handle()`
- Add feature test macro `__cpp_lib_fstream_native_handle`
- Fix errors with opening the file with POSIX APIs in Motivation (see, we need this paper, fstreams are easier to open correctly!)
- Add additional motivating use case in vectored/scatter-gather IO
- `Regular` -> `regular`

Incorporate LEWGI feedback from Cologne (July 2019):

- Move to a member function and member typedef
- Make `native_handle` return value not be mandated to be unique

- Add note about how the presence of the members is required, and not implementation-defined (like for thread)

LEWGI Polls from Cologne:

Attendance: 21

We should promise more committee time pursuing P1759, knowing that our time is scarce and this will leave less time for other work.

SF	F	N	A	SA
3	7	6	1	0

Knowing what we know now, we should promise more committee time to pursuing a unifying native handle type for the standard library, knowing that our time is scarce and this will leave less time for other work.

SF	F	N	A	SA
0	0	3	6	5

SA: Implementers had concerns with inability to add new native handle types without breaking ABI.

Member function (MF) vs free function (FF) added to stream classes.

SMF	MF	N	FF	SFF
2	9	4	1	1

SFF: Member function bad, free function good

The native handle type and native handle member function should always exist (they may return an invalid handle).

SF	F	N	A	SA
UNANIMOUS CONSENT				

An implementer had concerns how this paper wouldn't be useful if native handles were as underspecified as they are with `thread`.

Forward P1759, with a member function API that is always defined, to LEWG for C++23, knowing that our time is scarce and this will leave less time for other work.

SF	F	N	A	SA
3	10	2	2	0

§ 1.2. Revision 0

Initial revision.

§ 2. Overview

This paper proposes adding a new typedef to standard file streams: `native_handle_type`. This type is an alias to whatever type the platform uses for its file descriptors: `int` on POSIX, `HANDLE` (`void*`) on Windows, and something else on other platforms. This type is a non-owning handle and is to be small, `Regular`, standard layout, and trivially copyable.

Alongside this, this paper proposes adding a concrete member function: `.native_handle()`, returning a `native_handle_type`, to the following class templates:

- `basic_filebuf`
- `basic_ifstream`
- `basic_ofstream`
- `basic_fstream`

§ 3. Motivation

For some operations, using OS/platform-specific file APIs is necessary. If this is the case, they are unable to use iostreams without reopening the file with the platform-specific APIs.

For example, if someone wanted to query the time a file was last modified on POSIX, they'd use `::fstat`, which takes a file descriptor:

```
int fd = ::open("~/foo.txt", O_RDONLY);
::stat s{};
int err = ::fstat(fd, &s);
std::chrono::sys_seconds last_modified = std::chrono::seconds(s.st_mtime.tv
```

Note: The Filesystem TS introduced the `file_status` structure and `status` function retuning one. This doesn't solve our problem, because `std::filesystem::status` takes a path, not a native file descriptor (using paths is potentially racy), and `std::filesystem::file_status` only contains member functions `type()` and `permissions()`, not one for last time of modification. Extending this structure is out of scope for this proposal, and not feasible for every single possible operation the user may wish to do with OS APIs.

If the user needs to do a single operation not supported by the standard library, they have to make choice between only using OS APIs, or reopening the file every time necessary, likely forgetting to close the file, or running into buffering or synchronization issues.

```

// Writing the latest modification date to a file
std::chrono::sys_seconds last_modified(int fd) {
    // See above
}

// Today's code

// Option #1:
// Use iostreams by reopening the file
{
    int fd = ::open("~/foo.txt", O_RDONLY); // CreateFile on Windows
    auto lm = last_modified(fd);

    ::close(fd); // CloseFile on Windows
    // Hope the path still points to the file!
    // Need to allocate
    std::ofstream of("~/foo.txt");
    of << std::chrono::format("%c", lm) << '\n';
    // Need to flush
}

// Option #2:
// Abstain from using iostreams altogether
{
    int fd = ::open("~/foo.txt", O_RDWR);
    auto lm = last_modified(fd);

    // Using ::write() is clunky;
    // skipping error handling for brevity
    auto str = std::chrono::format("%c", lm);
    str.push_back('\n');
    ::write(fd, str.data(), str.size());
    // Remember to close!
    // Hope format or push_back doesn't throw
    ::close(fd);
}

// This proposal
// No need to use platform-specific APIs to open the file
{
    std::ofstream of("~/foo.txt");
    auto lm = last_modified(of.native_handle());
    of << std::chrono::format("%c", lm) << '\n';
}

```

```
// RAII does ownership handling for us
}
```

The utility of getting a file descriptor (or other native file handle) is not limited to getting the last modification date. Other examples include, but are definitely not limited to:

- file locking (`fcntl()` + `F_SETLK` on POSIX, `LockFile` on Windows)
- getting file status flags (`fcntl()` + `F_GETFL` on POSIX, `GetFileInformationByHandle` on Windows)
- vectored/scatter-gather IO (`vread()/vwrite()`)
- non-blocking IO (`fcntl()` + `O_NONBLOCK/F_SETSIG` on POSIX)

Basically, this paper would make standard file streams interoperable with operating system interfaces, making iostreams more useful in that regard.

An alternative would be adding a lot of this functionality to `fstream` and `filesystem`. The problem is, that some of this behavior is inherently platform-specific. For example, getting the inode of a file is something that only makes sense on POSIX, so cannot be made part of the `fstream` interface, and is only accessible through the native file descriptor.

Facilities replacing iostreams, although desirable, are not going to be available in the standard in the near future. The author, alongside many others, would thus find this functionality useful.

§ 4. Scope

This paper does *not* propose constructing a file stream or stream buffer from a native file handle. The author is worried of ownership and implementation issues possibly associated with this design.

```
// NOT PROPOSED
#include <fstream>
#include <fcntl.h>

auto fd = ::open(/* ... */);
auto f = std::fstream{fd};
```

This paper also does not touch anything related to `FILE*`.

§ 5. Design Discussion

See polls in [§1 Revision History](#) for more details

§ 5.1. Type of `native_handle_type`

In this paper, the definition for `native_handle_type` is *much* more strict than in `thread`. For reference, this is the wording from 32.2.3 *Native handles* [**thread.req.native**], from [\[N4800\]](#):

Several classes described in this Clause have members `native_handle_type` and `native_handle`. The presence of these members and their semantics is implementation-defined. [*Note*: These members allow implementations to provide access to implementation details. Their names are specified to facilitate portable compile-time detection. Actual use of these members is inherently non-portable. — *end note*]

During the review of R0 of this paper in Cologne by LEWGI, an implementor mentioned how having the same specification here would make this paper effectively useless. Having the presence of a member be implementation-defined without a feature detect macro was deemed as bad design, which should not be replicated in this paper.

The proposed alternative in this paper, as directed by LEWGI, is allowing a conforming implementation to return an invalid native file handle, if it cannot be retrieved.

§ 5.2. Member function vs free function

R0 of this paper had a free function `std::native_file_handle()` instead of a member function, due to possible ABI-related concerns. This turned out not to be an issue, so the design was changed to be a member function, for consistency with `thread`.

§ 5.3. Precondition

The member function `.native_handle()`, as specified in this paper, has a precondition of `.is_open() == true`. The precondition is specified with "Expects", so breaking it would be UB, and in practice enforced with an assert. An alternative to this would be throwing if the file is not open, or returning some unspecified invalid handle.

§ 6. Impact On the Standard and Existing Code

This proposal is a pure library extension, requiring no changes to the core language. It would cause no existing conforming code to break.

§ 7. Implementation

Implementing this paper should be a relatively trivial task.

Although all implementations surveyed (libstdc++, libc++ and MSVC) use `FILE*` instead of native file descriptors in their `basic_filebuf` implementations, these platforms provide facilities to get a native handle from a `FILE*`; `fileno` on POSIX, and `_fileno` + `_get_osfhandle` on Windows. The following reference implementations use these.

For libstdc++ on Linux:

```
template <class CharT, class Traits>
class basic_filebuf {
    // ...
    using native_handle_type = int;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // _M_file (__basic_file<char>) has a member function for this purpose
        return _M_file.fd();
        // ::fileno(file.file()) could also be used
    }
    // ...
}

// (i|o)fstream implementation is trivial with rdbuf()
```

For libc++ on Linux:

```

template <class CharT, class Traits>
class basic_filebuf {
    // ...
    using native_handle_type = int;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // __file_ is a FILE*
        return ::fileno(__file_)
    }
    // ...
}

// (i|o)fstream implementation is trivial with rdbuf()

```

For MSVC:

```

template <class CharT, class Traits>
class basic_filebuf {
    // ...
    using native_handle_type = HANDLE;
    // ...
    native_handle_type native_handle() {
        assert(is_open());
        // _Myfile is a FILE*
        auto cfile = ::_fileno(_Myfile);
        // _get_osfhandle returns intptr_t, HANDLE is a void*
        return static_cast<HANDLE>(::_get_osfhandle(cfile));
    }
    // ...
}

// (i|o)fstream implementation is trivial with rdbuf()

```

§ 8. Prior Art

[Boost.IOStreams] provides `file_descriptor`, `file_descriptor_source`, and `file_descriptor_sink`, which, when used in conjunction with `stream_buffer`, are

`std::basic_streambuf`s using a file descriptor. These classes can be constructed from a path or a native handle (`int` or `HANDLE`) and can also return it with member function `handle()`.

The Networking TS [N4734] has members `native_handle_type` and `.native_handle()` in numerous places, including `std::net::socket`. It specifies (in `[socket.reqmts.native]`) the presence of these members in a similar fashion to `thread`, as in making their presence implementation-defined. It does, however, recommend POSIX-based systems to use `int` for this purpose.

Niall Douglas's [P1031] also defined a structure `native_handle_type` with an extensive interface and a member `union` with an `int` and a `HANDLE`, with a constructor taking either one of these.

§ 8.1. Discussion

There has been some discussion over the years about various things relating to this issue, but as far as the author is aware, no concrete proposal has ever been submitted.

There have been a number of threads on std-discussion and std-proposals:

[\[std-proposals-native-handle\]](#), [\[std-discussion-fd-io\]](#), [\[std-proposals-native-raw-io\]](#), [\[std-proposals-fd-access\]](#). The last one of these lead to a draft paper, that was never submitted: [\[access-file-descriptors\]](#).

The consensus that the author took from these discussions is, that native handle support for iostreams would be very much welcome.

An objection was raised by Billy O'Neal to being able to retrieve a native file handle from a standard file stream:

[This] also would need to mandate that the C++ streams be implemented directly such that there was a 1:1 native handle relationship, which may not be the case. For instance, a valid implementation of C++ iostreams would be on top of `cstdio`, which would not have any kind of native handle to expose.

– Billy O'Neal: [\[std-proposals-billy-oneal\]](#)

Every implementation surveyed did implement `basic_filebuf` on top of C `stdio`, but these platforms also provide functionality for getting a file descriptor out of a `FILE*`. On every platform, file I/O is ultimately implemented on top of native APIs, so not providing access to a file descriptor from a `FILE*` would be rather unfortunate. Should such a platform exist, they probably don't have a conforming C++ implementation anyway. See [§7 Implementation](#) for more.

Additionally, as directed by LEWGI, `.native_handle()` can just return an invalid handle, if the implementation really can't get a valid one corresponding to the file.

§ 8.2. Existing precedent for presence of `native_handle`

Types <i>with</i> a standard way of getting the native handle	Types <i>without</i> a standard way of getting the native handle
<code>std::thread</code> <code>std::mutex</code> - Networking TS [N4734] types (e.g. <code>std::net::socket</code>) Proposals: - Low level file I/O [P1031]	<code>std::fstream</code>/<code>std::filebuf</code> <code>FILE*</code>

§ 9. Technical Specifications

The wording is based on [\[N4800\]](#).

Add the following row into *Table 36: Standard library feature-test macros* [**tab:support.ft**] in [**support.limits.general**]:

<code>__cpp_lib_filesystem</code>	201703L	<code><filesystem></code>
<u><code>__cpp_lib_fstream_native_handle</code></u>	<u><i>TBD</i></u>	<u><code><fstream></code></u>
<code>__cpp_lib_gcd_lcm</code>	201606L	<code><numeric></code>

Add the following subsection (?) into *File-based streams* [**file.streams**], after [**fstream.syn**].

Note to editor: Replace ? with the appropriate section number.

§ ? Native handles [file.native]

Several classes described in this section have a member `native_handle_type`.

The type `native_handle_type` serves as a type representing a platform-specific handle to a file. It satisfies the requirements of `regular`, and is trivially copyable and standard layout.

[*Note:* For operating systems based on POSIX, `native_handle_type` should be `int`. For Windows-based operating systems, it should be `HANDLE`. — *end note*]

Add the following into *Class template* `basic_filebuf` [filebuf]:

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_filebuf : public basic_streambuf<charT, traits> {
    public:
        // Note: Add after other member typedefs
        using native_handle_type = implementation-defined; // see [file.

        // Note: Add as the last [filebuf.members]
        native_handle_type native_handle();

        // ...
    }
}
```

Modify paragraph § 1 of *Class template* `basic_filebuf` [filebuf]:

The class `basic_filebuf<charT, traits>` associates both the input sequence and the output sequence with a file. The file has an associated `native_handle_type`. Whether the associated `native_handle_type` is unique for each instance of a `basic_filebuf`, is implementation-defined. [*Note:* This differs from thread native handles [thread.req.native], the presence of which is implementation-defined. — *end note*]

Add the following to the end of *Member functions* [filebuf.members]:

```
native_handle_type native_handle();
```

Expects: `is_open()` is `true`.

Throws: Nothing.

Returns: The `native_handle_type` associated with the underlying file of `*this`.

Add the following into Class template `basic_ifstream` [**ifstream**]:

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ifstream : public basic_istream<charT, traits> {
    public:
        // Note: Add after other member typedefs
        using native_handle_type =
            typename basic_filebuf<charT, traits>::native_handle_type;

        // Note: Add as the last [ifstream.members]
        native_handle_type native_handle();

        // ...
    }
}
```

Add the following to the end of Member functions [**ifstream.members**]:

```
native_handle_type native_handle();
```

Effects: Equivalent to: `return rdbuf()->native_handle();`.

Add the following into Class template `basic_ofstream` [**ofstream**]:

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ofstream : public basic_ostream<charT, traits> {
    public:
        // Note: Add after other member typedefs
        using native_handle_type =
            typename basic_filebuf<charT, traits>::native_handle_type;

        // Note: Add as the last [ofstream.members]
        native_handle_type native_handle();

        // ...
    }
}

```

Add the following to the end of *Member functions* [**ofstream.members**]:

```

native_handle_type native_handle();

```

Effects: Equivalent to: `return rdbuf()->native_handle();`.

Add the following into *Class template* `basic_fstream` [**fstream**]:

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_fstream : public basic_istream<charT, traits> {
    public:
        // Note: Add after other member typedefs
        using native_handle_type =
            typename basic_filebuf<charT, traits>::native_handle_type;

        // Note: Add as the last [fstream.members]
        native_handle_type native_handle();

        // ...
    }
}

```

Add the following to the end of *Member functions* [**fstream.members**]:

```

native_handle_type native_handle();

```

Effects: Equivalent to: `return rdbuf()->native_handle();`.

§ 10. Acknowledgements

Thanks to Niall Douglas for feedback, encouragement and ambitious suggestions for this paper.

Thanks to the rest of the co-authors of [\[P1750\]](#) for the idea after cutting this functionality out, especially to Jeff Garland for providing a heads-up about a possible ABI-break that I totally would've missed, even though it ended up being a non-issue.

§ References

§ Informative References

[ACCESS-FILE-DESCRIPTORS]

Bruce S. O. Adams. [file streams and access to the file descriptor](https://docs.google.com/viewer?a=v&pid=forums&srcid=MTEwODAzNzI2MjM1OTc0MjE3MjkBMDY0OTY1OTUzMjAwNzY0MTA0MjkBakhWMHBFLUNGd0FKATAuMQFpc29jcHAub3JnAXYy&authuser=0). URL: <https://docs.google.com/viewer?a=v&pid=forums&srcid=MTEwODAzNzI2MjM1OTc0MjE3MjkBMDY0OTY1OTUzMjAwNzY0MTA0MjkBakhWMHBFLUNGd0FKATAuMQFpc29jcHAub3JnAXYy&authuser=0>

[Boost.IOStreams]

Jonathan Turkanis. [Boost.IOStreams](https://www.boost.org/doc/libs/1_70_0/libs/iostreams/doc/index.html). URL: https://www.boost.org/doc/libs/1_70_0/libs/iostreams/doc/index.html

[N4734]

Jonathan Wakely. [Working Draft, C ++ Extensions for Networking](https://wg21.link/n4734). 4 April 2018. URL: <https://wg21.link/n4734>

[N4800]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4800). 21 January 2019. URL: <https://wg21.link/n4800>

[P1031]

Niall Douglas. [Low level file i/o library](https://wg21.link/p1031). URL: <https://wg21.link/p1031>

[P1750]

Klemens Morgenstern, Jeff Garland, Elias Kosunen, Fatih Bakir. [A Proposal to Add Process Management to the C++ Standard Library](https://wg21.link/p1750). URL: <https://wg21.link/p1750>

[STD-DISCUSSION-FD-IO]

[File descriptor-backed I/O stream?](https://groups.google.com/a/isocpp.org/forum/#!topic/std-discussion/macDvhFDjrjU). URL: <https://groups.google.com/a/isocpp.org/forum/#!topic/std-discussion/macDvhFDjrjU>

[STD-PROPOSALS-BILLY-ONEAL]

Billy O'Neal. [Comment on 'native_handle for basic_filebuf'](https://groups.google.com/a/isocpp.org/d/msg/std-proposals/oCEErQbI9sM/rMkAMOkxFvMJ). URL: <https://groups.google.com/a/isocpp.org/d/msg/std-proposals/oCEErQbI9sM/rMkAMOkxFvMJ>

[STD-PROPOSALS-FD-ACCESS]

[file streams and access to the file descriptor](https://groups.google.com/a/isocpp.org/d/topic/std-proposals/XcQ4FZJKDbM/discussion). URL: <https://groups.google.com/a/isocpp.org/d/topic/std-proposals/XcQ4FZJKDbM/discussion>

[STD-PROPOSALS-NATIVE-HANDLE]

[native_handle for basic_filebuf](https://groups.google.com/a/isocpp.org/d/topic/std-proposals/oCEErQbI9sM/discussion). URL: <https://groups.google.com/a/isocpp.org/d/topic/std-proposals/oCEErQbI9sM/discussion>

[STD-PROPOSALS-NATIVE-RAW-IO]

Native raw IO and FILE* wrappers?. URL: <https://groups.google.com/a/isocpp.org/d/topic/std-proposals/Q4RdFSZggSE/discussion>